

Module 8: Building and Deploying End-to-End GenAI Applications

Demo 1

Demo: Complete GenAI Application Development Guide - Practical Implementation

Overview

This document provides a comprehensive practical implementation guide for developing, deploying, and monitoring a production-ready GenAI application. The implementation demonstrates the complete lifecycle of AI application development from local development to cloud deployment, incorporating industry best practices for FastAPI integration, LangChain conversation management, GCP Cloud Run deployment, and LangSmith observability.

Scenario

TechConsult Solutions is a fast-growing technology consulting firm known for delivering top-tier AI implementation services across industries like healthcare, finance, and retail. With 200+ clients and a team of 50+ consultants, the company has built a solid reputation as a trusted partner in the AI transformation space.

As client demand increases across time zones and technical domains, their current support system is becoming a bottleneck. To scale effectively while maintaining service quality, leadership is exploring an AI-powered assistant that can offer 24/7 support—handling routine queries, offering initial guidance, and escalating complex issues to human consultants as needed.

Problem Statement

TechConsult Solutions faces several critical challenges in scaling their client support operations:

1. Scalability Bottlenecks

- **Limited Support Hours:** Current support is available only during business hours, leaving international clients without immediate assistance
- **Resource Constraints:** With 200+ clients and growing, the current team of 5 support engineers cannot handle the increasing volume of inquiries

2. Knowledge Management Challenges

- **Inconsistent Information:** Different support engineers may provide varying answers to similar questions, leading to confusion and potential implementation errors
- **Knowledge Silos:** Critical expertise is often concentrated in specific team members, creating dependencies and potential knowledge gaps

3. Operational Inefficiencies

- **Repetitive Query Handling:** Approximately 60% of client inquiries are repetitive questions about common AI implementation challenges
- **Manual Triage Process:** All inquiries require manual review and routing, consuming significant administrative time

4. Quality Assurance Issues

- **No Conversation Tracking:** Limited ability to track and analyze support conversations for quality improvement
- **Missing Performance Metrics:** Lack of detailed analytics on response effectiveness, client satisfaction, and support agent performance

5. Technology Infrastructure Limitations

- **No Observability:** Current system lacks comprehensive monitoring and logging capabilities
- **Limited Scalability:** Existing infrastructure cannot easily scale to handle increasing demand

Solution

To address these challenges, TechConsult Solutions will implement a comprehensive AI-powered technical support system using modern cloud-native technologies and industry best practices.

Step-by-Step Approach:

This guide will walk you through building a complete GenAI chatbot application that demonstrates:

- **FastAPI Integration:** Creating REST APIs with proper request/response handling
- **LangChain Integration:** Using LangChain for conversation management with Gemini AI
- **Session Management:** Maintaining conversation history across multiple requests
- **Cloud Deployment:** Deploying to Google Cloud Run with containerization
- **Monitoring:** Using LangSmith for request tracing and performance monitoring

Prerequisites

Software Requirements



- **Python 3.11+** installed on your system
- **VS Code** (recommended IDE)
- **Git** for version control
- **Google Chrome** or any modern web browser

Accounts Needed

- **Google Account** (for Google Cloud Platform and Gemini API)
- **LangSmith Account**

Environment Setup

Step 1: Create Project Directory

Open VS Code and create a new project directory.

Create the main project directory

```
mkdir <folder_name>
```

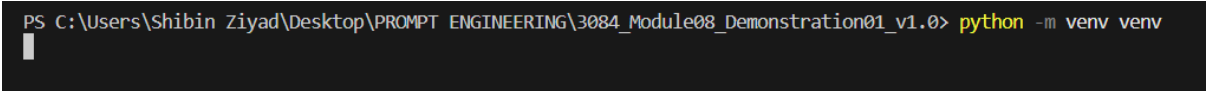
```
cd <folder_name>
```

Creates a dedicated folder for your project to keep everything organized.

Step 2: Create Python Virtual Environment

Create virtual environment

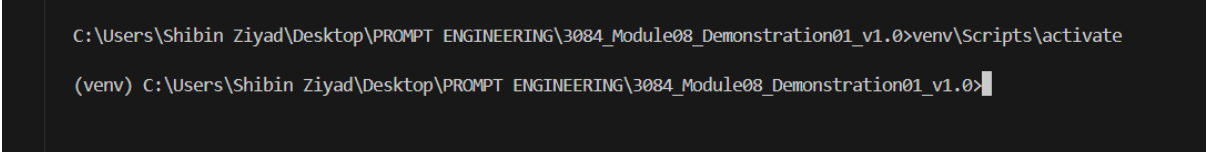
```
python -m venv venv
```



```
PS C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084_Module08_Demonstration01_v1.0> python -m venv venv
```

Activate virtual environment (Windows)

```
venv\Scripts\activate
```



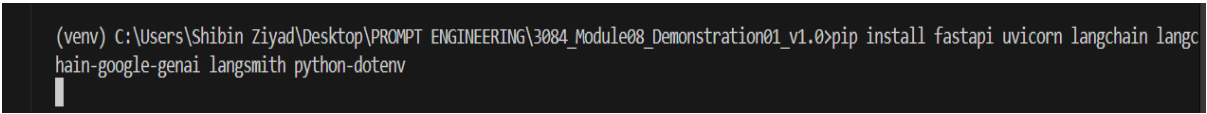
```
C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084_Module08_Demonstration01_v1.0> venv\Scripts\activate  
(venv) C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084_Module08_Demonstration01_v1.0>
```

Creates an isolated Python environment to avoid conflicts with other projects. The virtual environment ensures your project dependencies don't interfere with your system Python.

Step 3: Install Required Dependencies

Install all required packages

```
pip install fastapi uvicorn langchain langchain-google-genai langsmith python-dotenv
```



```
(venv) C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084_Module08_Demonstration01_v1.0> pip install fastapi uvicorn langchain langchain-google-genai langsmith python-dotenv
```

What each package does:

- **fastapi**: Web framework for building APIs
- **uvicorn**: ASGI server to run FastAPI applications
- **langchain**: Framework for building LLM applications
- **langchain-google-genai**: Google Gemini integration for LangChain
- **langsmith**: Monitoring and tracing for LangChain applications
- **python-dotenv**: Load environment variables from .env files

Step 4: Create Requirements File

Generate requirements.txt file

```
pip freeze > requirements.txt
```

What this does: Creates a file listing all installed packages and their versions. This is essential for deployment and sharing your project.

Project Structure Creation

Step 5: Create Project Structure

Create app directory for application code

```
mkdir app
```

Create empty __init__.py file

```
echo. > app\__init__.py
```

Create other necessary files

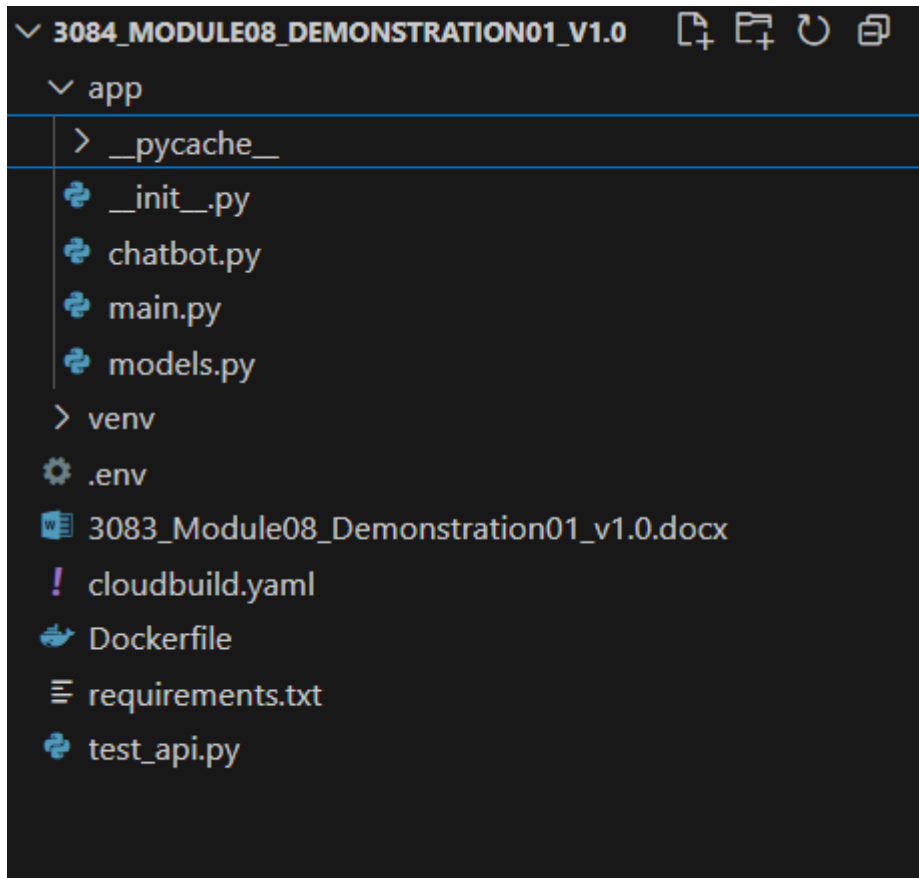
```
echo. > .env
```

```
echo. > Dockerfile
```

```
echo. > test_api.py
```

Creates the proper Python package structure. The app directory will contain your application code, and `__init__.py` makes it a Python package. Similarly create all the python files.

Step 6: Verify Project Structure



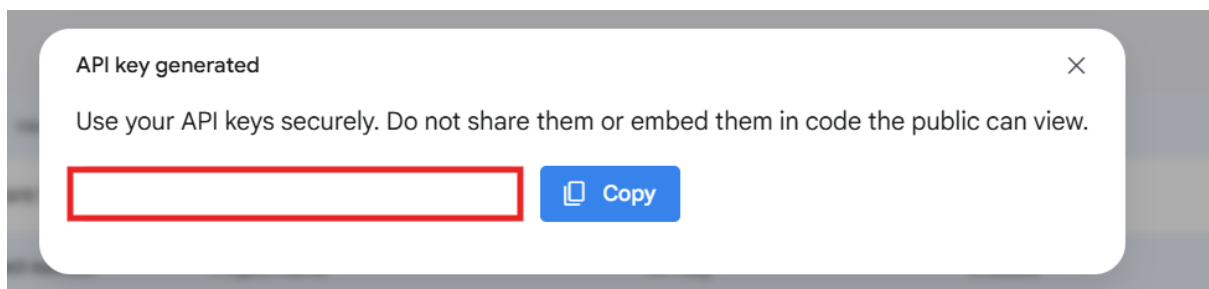
API Keys Configuration

Step 7: Get Gemini API Key

1. **Open Google AI Studio:** Go to <https://makersuite.google.com/app/apikey>
2. **Sign in** with your Google account
3. **Create API Key:** Click "Create API Key"



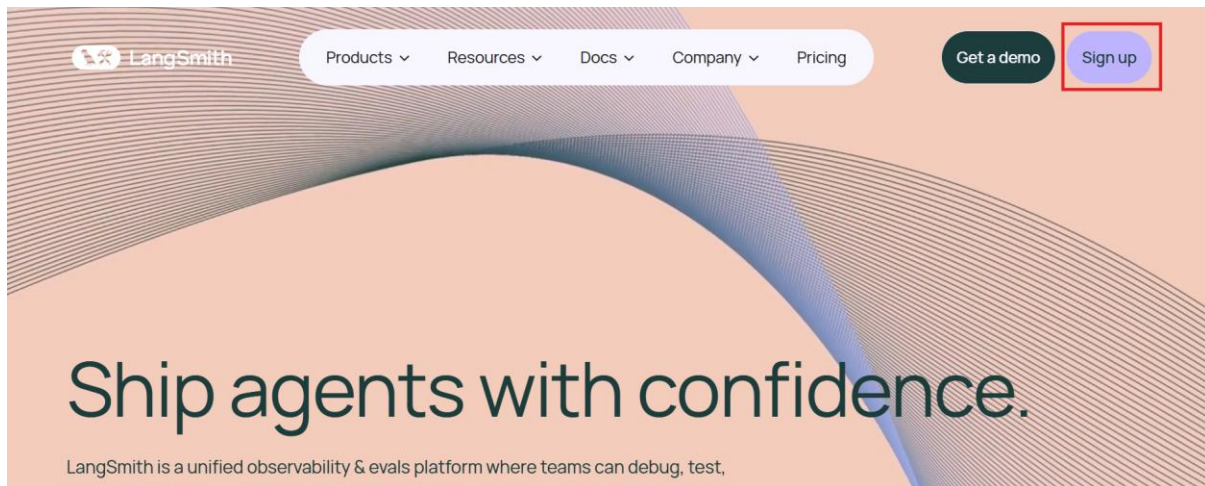
4. **Copy the key:** Save it securely (you'll need it in the next step)



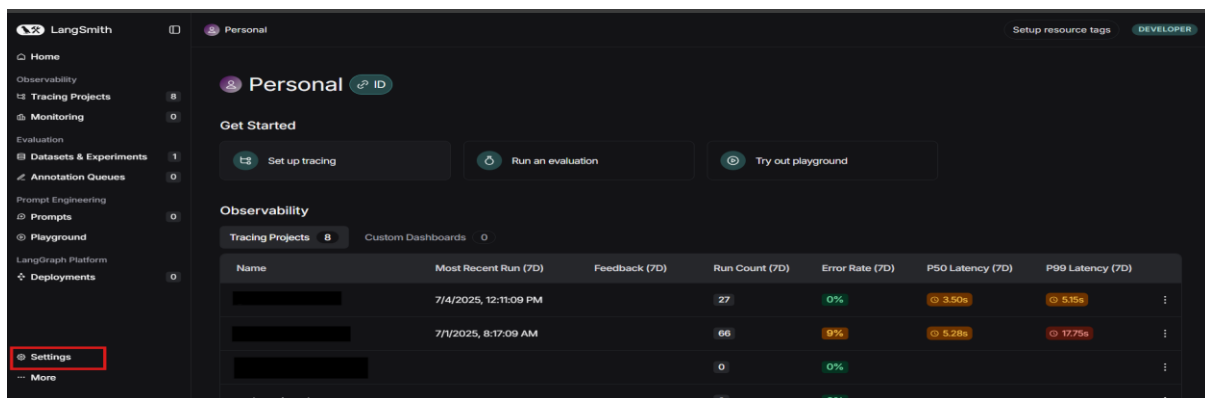
Provides access to Google's Gemini AI model for generating chatbot responses.

Step 8: Get LangSmith API Key

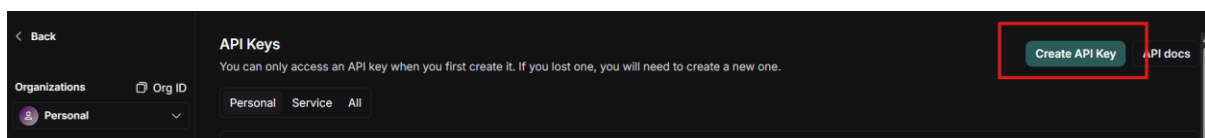
1. **Open LangSmith:** Go to <https://smith.langchain.com/>
2. **Sign up** for a free account



3. **Navigate to Settings:** Click on your profile → Settings



4. **Create API Key:** Go to "API Keys" tab → "Create API Key"



5. **Copy the key:** Save it securely

Step 9: Configure Environment Variables

Open the .env file in VS Code and add:

```
1 GOOGLE_API_KEY=your_gemini_api_key_here
2 LANGCHAIN_TRACING_V2=true
3 LANGCHAIN_ENDPOINT=https://api.smith.langchain.com
4 LANGCHAIN_API_KEY=your_langsmith_api_key_here
5 LANGCHAIN_PROJECT=genai-chatbot-demo
```

What each variable does:

- **GOOGLE_API_KEY:** Your Gemini API key for AI responses
- **LANGCHAIN_TRACING_V2:** Enables LangSmith tracing
- **LANGCHAIN_ENDPOINT:** LangSmith API endpoint
- **LANGCHAIN_API_KEY:** Your LangSmith API key
- **LANGCHAIN_PROJECT:** Project name in LangSmith dashboard

Code Implementation

Step 10: Create Pydantic Models

- **Create app/models.py:**
- Defines the data structures (schemas) for your API. Pydantic provides automatic validation and serialization.

Step 11: Create Chatbot Service

- **Create app/chatbot.py:**
- Implements the core chatbot functionality using LangChain and Gemini AI. Manages conversation sessions and generates responses.

Step 12: Create FastAPI Application

- **Create app/main.py:**
- This is the main FastAPI application

- Creates the web API endpoints that clients will interact with. Handles HTTP requests and responses.

Step 13: Create Test Script

- **Create test_api.py in the root directory:**
- Copy the test_api.py content from the previous artifacts
- This script tests all API endpoints
- Provides automated testing for your API endpoints to ensure everything works correctly.

Step 14: Create Dockerfile and cloudbuild.yaml

- **Create Dockerfile in the project root:**
- Copy the Dockerfile content from the previous artifacts
- This defines how to build the container

Local Testing

Step 15: Start the Development Server



Start the FastAPI development server

```
uvicorn app.main:app --reload
```

Starts your API server on <http://localhost:8000>. The `--reload` flag automatically restarts the server when you make code changes.

```
(venv) C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084_Module08_Demonstration01_v1.0>uvicorn app.main:app --reload
INFO: Will watch for changes in these directories: [C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084_Module08_Demonstration01_v1.0]
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [18060] using StatReload
INFO: Started server process [40036]
INFO: Waiting for application startup.
INFO:app.main:Starting GenAI Chatbot API...
INFO:app.chatbot:Chatbot service initialized successfully
INFO:app.main:Chatbot service initialized
INFO: Application startup complete.
```

Step 16: Test API Documentation

1. **Open browser:** Go to <http://localhost:8000/docs>

GenAI Chatbot API 1.0.0 OAS 3.1

/openapi.json

A simple chatbot API using FastAPI, LangChain, and Gemini

default		^
GET	/health Health Check	▼
POST	/chat Chat	▼
DELETE	/session/{session_id} Clear Session	▼
GET	/session/{session_id} Get Session Info	▼
GET	/ Root	▼

2. **Explore API:** You'll see interactive API documentation

3. **Test endpoints:** Try the **/health** endpoint first

```

Pretty-print ☒
{
  "status": "healthy",
  "message": "GenAI Chatbot API is running"
}

```

FastAPI automatically generates interactive API documentation that you can use to test your endpoints.

Step 17: Test Health Endpoint

Open a new terminal in VS Code (keep the server running) and run:

Test the health endpoint

```
curl -X GET http://localhost:8000/health
```

```
C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084_Module08_Demonstration01_v1.0>curl -X GET "http://localhost:8000/health"
```

```

C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084_Module08_Demonstration01_v1.0>curl -X GET "http://localhost:8000/health"
{"status": "healthy", "message": "GenAI Chatbot API is running"}
C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084_Module08_Demonstration01_v1.0>

```

Verifies that your API is running and responding correctly.

Step 18: Test Chat Endpoint

Test the chat endpoint with a simple message

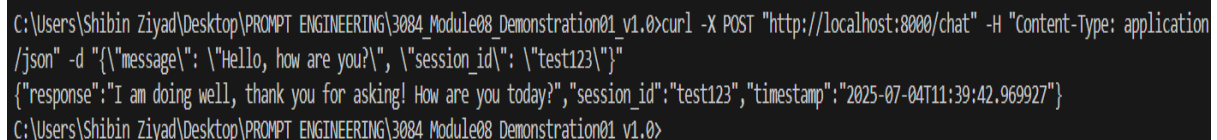
```
url -X POST "http://localhost:8000/chat" \  
    -H "Content-Type: application/json" \  
    -d "{\"message\": \"Hello, how are you?\", \  
    \"session_id\": \"test123\"}"
```

Test session info

```
curl -X GET "http://localhost:8000/session/test123"
```

Clear session

```
curl -X DELETE "http://localhost:8000/session/test123"
```



```
C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084 Module08 Demonstration01 v1.0>curl -X POST "http://localhost:8000/chat" -H "Content-Type: application  
/json" -d "{\"message\": \"Hello, how are you?\", \"session_id\": \"test123\"}"  
{\"response\": \"I am doing well, thank you for asking! How are you today?\", \"session_id\": \"test123\", \"timestamp\": \"2025-07-04T11:39:42.969927\"}  
C:\Users\Shibin Ziyad\Desktop\PROMPT ENGINEERING\3084 Module08 Demonstration01 v1.0>
```

Tests the core chatbot functionality by sending a message and receiving an AI-generated response. (If you are running in VS code, it has to be in a single line).

Step 19: Run Complete Test Suite

Run the automated test script

```
python test_api.py
```

```

| **Purpose** | To simulate human intelligence in machines. | To enable machines to learn from data without explicit
programming. |
| **Relationship** | ML is a subset of AI. | AI is the broader field that ML contributes to.
|
**In simpler terms:**

All machine learning is AI, but not all AI is machine learning. AI is the umbrella, and ML is one of the tools under that umbrella. Think of it like squares and rectangles: All squares are rectangles, but not all rectangles are squares.
Session: test_session_123
Timestamp: 2025-07-04T11:32:54.784544
✅ Chat tests passed

3. Testing Session Info...

Session Info: 200
Response: {'session_id': 'test_session_123', 'message_count': 8, 'last_activity': '2025-07-04T11:32:57.830465'}
✅ Session info test passed

4. Testing Clear Session...

Clear Session: 200
Response: {'message': 'Session test_session_123 cleared successfully'}
✅ Clear session test passed

5. Verifying Session Cleared...

Session Info: 200
Response: {'session_id': 'test_session_123', 'message_count': 0, 'last_activity': '2025-07-04T11:33:01.916717'}
✅ Session verification passed

🎉 All tests passed successfully!

```

Runs all automated tests to ensure all endpoints work correctly and conversation memory is functioning.

Google Cloud Platform Setup

Step 20: Install Google Cloud CLI



1. **Download:** Go to <https://cloud.google.com/sdk/docs/install>
2. **Download Windows installer** (or appropriate version for your OS)

Google Cloud Documentation Technology areas Cross-product tools Related sites Search English Console Sign in

Cloud SDK Guides Reference Resources

Filter

Discover
gcloud CLI overview
Cloud Client Libraries overview
Client libraries explained

Get started
Install the Google Cloud CLI
Other CLI installation methods
Recommended installation
Other installation methods

Authenticate to Google Cloud
Authentication methods
Ways to authenticate
Application Default Credentials
API keys
Get an ID token
Token types
Multi-factor authentication

Installation instructions

These instructions are for installing the Google Cloud CLI. For information about installing additional components, such as gcloud CLI commands at the alpha or beta release level, see [Managing gcloud CLI components](#).

★ Note: If you are behind a proxy or firewall, see the [proxy settings](#) page for more information on installation.

Linux Debian/Ubuntu Red Hat/Fedora/CentOS macOS **Windows**

The Google Cloud CLI works on Windows 8.1 and later and Windows Server 2012 and later.

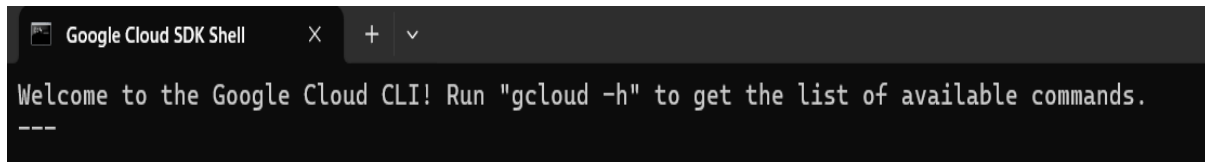
1. Download the **Google Cloud CLI installer**.

Alternatively, open a PowerShell terminal and run the following PowerShell commands:

```
(New-Object Net.WebClient).DownloadFile("https://dl.google.com/dl/cloudsdk/channels/rapid/downloads/google-cloud-sdk-  
& $env:Temp\GoogleCloudSDKInstaller.exe
```

On this page
Installation instructions
Other installation options
Manage an installation
Earlier versions of the gcloud CLI
Supported Python versions

3. **Run installer:** Follow the installation wizard
4. **Restart VS Code:** Close and reopen VS Code after installation
5. Open **Google Cloud SDK Shell**



Installs the command-line tools needed to interact with Google Cloud services.

Step 21: Verify Installation

Check if gcloud is installed

```
gcloud --version
```

```
C:\Users\Shibin Ziyad\AppData\Local\Google\Cloud SDK>gcloud --version
Google Cloud SDK 529.0.0
bq 2.1.19
core 2025.06.27
gcloud-crc32c 1.0.0
gsutil 5.35
```

Confirms that Google Cloud CLI is properly installed.

Step 22: Initialize Google Cloud CLI

Initialize gcloud - **gcloud init**

```
- gcloud auth login
```

```
C:\Users\Shibin Ziyad\AppData\Local\Google\Cloud SDK>gcloud init
Welcome! This command will take you through the configuration of gcloud.

Settings from your current configuration [default] are:
accessibility:
  screen_reader: 'False'
core:
  account:
  disable_usage_reporting: 'True'

Pick configuration to use:
[1] Re-initialize this configuration [default] with new settings
[2] Create a new configuration
```

What this does:

- Authenticates your Google account
- Sets up default project and region
- Configures your local environment to work with Google Cloud

Step 23: Create New Project (if needed)

- Create a new project (replace with your preferred project ID)
- Set your project
- `gcloud config set project YOUR_PROJECT_ID`

Step 24: Enable Required APIs

Enable Cloud Build API (for building containers)

- `gcloud services enable cloudbuild.googleapis.com`

Enable Cloud Run API (for deploying containers)

- `gcloud services enable run.googleapis.com`

Enable Container Registry API (for storing container images)

- `gcloud services enable containerregistry.googleapis.com`

```
C:\Users\Shibin Ziyad\AppData\Local\Google\Cloud SDK>gcloud services enable cloudbuild.googleapis.com  
C:\Users\Shibin Ziyad\AppData\Local\Google\Cloud SDK>gcloud services enable run.googleapis.com  
C:\Users\Shibin Ziyad\AppData\Local\Google\Cloud SDK>gcloud services enable containerregistry.googleapis.com
```

Enables the Google Cloud services needed for containerization and deployment.

Cloud Deployment

Step 25: Build and Push Container Image

Make sure you're in the project directory

- `cd genai-chatbot-demo`

Build and push (replace YOUR_PROJECT_ID with your actual project ID)

- `gcloud builds submit --tag gcr.io/YOUR_PROJECT_ID/genai-chatbot`

What this does:

- Builds your Docker container in Google Cloud
- Pushes the image to Google Container Registry
- Makes the image available for deployment

Expected output:



ID	CREATE_TIME	DURATION	SOURCE	STATUS
973a1208-f0bf-4d22-beef-09d3b4f49e51	2025-07-04T06:30:51+00:00	59S	gcr.io/hallowed-moment-464819-s7/genai-chatbot (+1 more)	build/source/175
1610568.231739	1.tgz			SUCCESS

Step 26: Deploy to Cloud Run

Deploy (replace YOUR_PROJECT_ID and API keys with actual values)

```
gcloud run deploy genai-chatbot \  
  
--image gcr.io/YOUR_PROJECT_ID/genai-chatbot \  
  
--platform managed \  
  
--region us-central1 \  
  
--allow-unauthenticated \
```

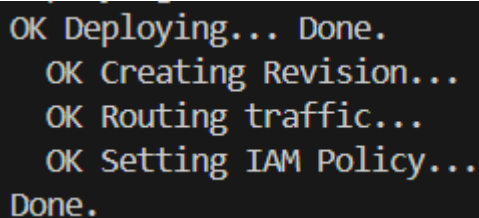
```
--set-env-vars  
GOOGLE_API_KEY=your_actual_gemini_key,LANGCHAIN_TRACING_V  
2=true,LANGCHAIN_ENDPOINT=https://api.smith.langchain.com  
,LANGCHAIN_API_KEY=your_actual_langsmith_key,LANGCHAIN_PR  
OJECT=genai-chatbot-demo
```

What each parameter does:

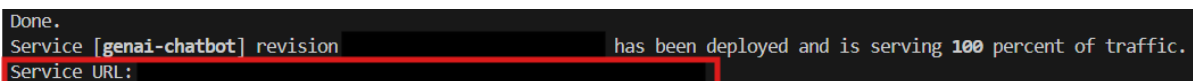
- **--image:** Specifies the container image to deploy
- **--platform managed:** Uses fully managed Cloud Run
- **--region:** Deployment region (choose closest to your users)
- **--allow-unauthenticated:** Allows public access to your API
- **--set-env-vars:** Sets environment variables in the cloud

Expected output:

Deploying container to Cloud Run service [genai-chatbot]...



```
OK Deploying... Done.  
OK Creating Revision...  
OK Routing traffic...  
OK Setting IAM Policy...  
Done.
```



```
Done.  
Service [genai-chatbot] revision [redacted] has been deployed and is serving 100 percent of traffic.  
Service URL: [redacted]
```

Testing Deployed Application

Step 27: Test Deployed Health Endpoint

Test the health endpoint (replace with your actual URL)

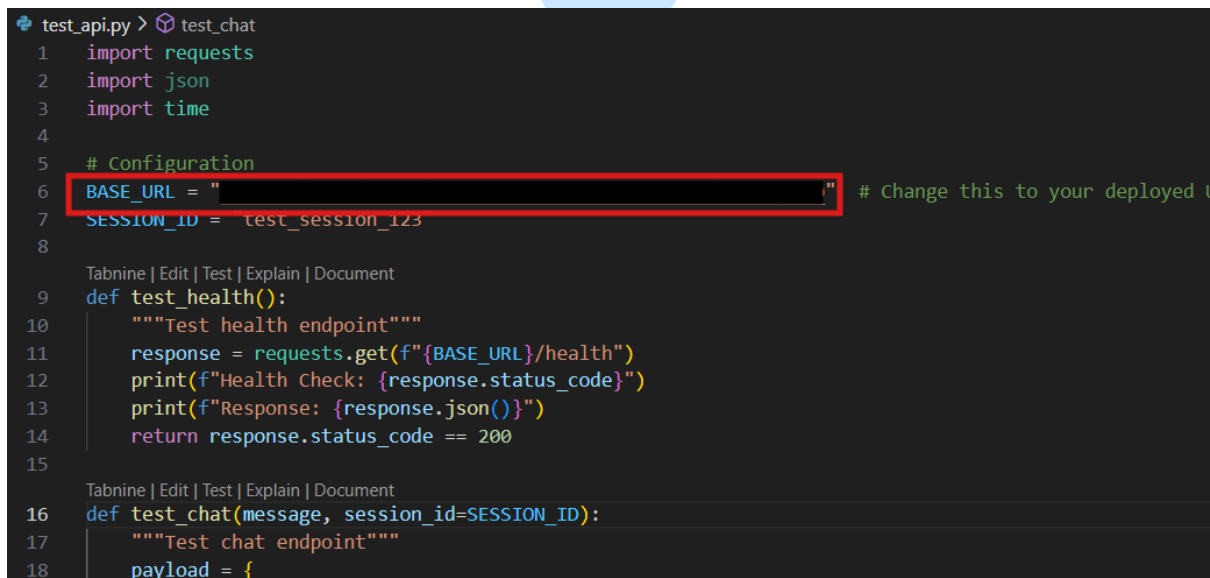
```
curl -X GET "https://genai-chatbot-xxxxx-uc.a.run.app/health"
```

```
Pretty-print ☒
{
  "message": "Welcome to GenAI Chatbot API",
  "version": "1.0.0",
  "endpoints": {
    "health": "/health",
    "chat": "/chat",
    "docs": "/docs"
  }
}
```

Verifies that your deployed application is accessible and working.

Step 28: Update Test Script for Cloud Testing

1. Open `test_api.py` in VS Code
2. Update the `BASE_URL`:
3. `BASE_URL = "https://genai-chatbot-xxxxxx-uc.a.run.app" # Your actual URL`



```
test_api.py > test_chat
1 import requests
2 import json
3 import time
4
5 # Configuration
6 BASE_URL = "https://genai-chatbot-xxxxxx-uc.a.run.app" # Change this to your deployed URL
7 SESSION_ID = test_session_123
8
9 def test_health():
10     """Test health endpoint"""
11     response = requests.get(f"{BASE_URL}/health")
12     print(f"Health Check: {response.status_code}")
13     print(f"Response: {response.json()}")
14     return response.status_code == 200
15
16 def test_chat(message, session_id=SESSION_ID):
17     """Test chat endpoint"""
18     payload = {
```

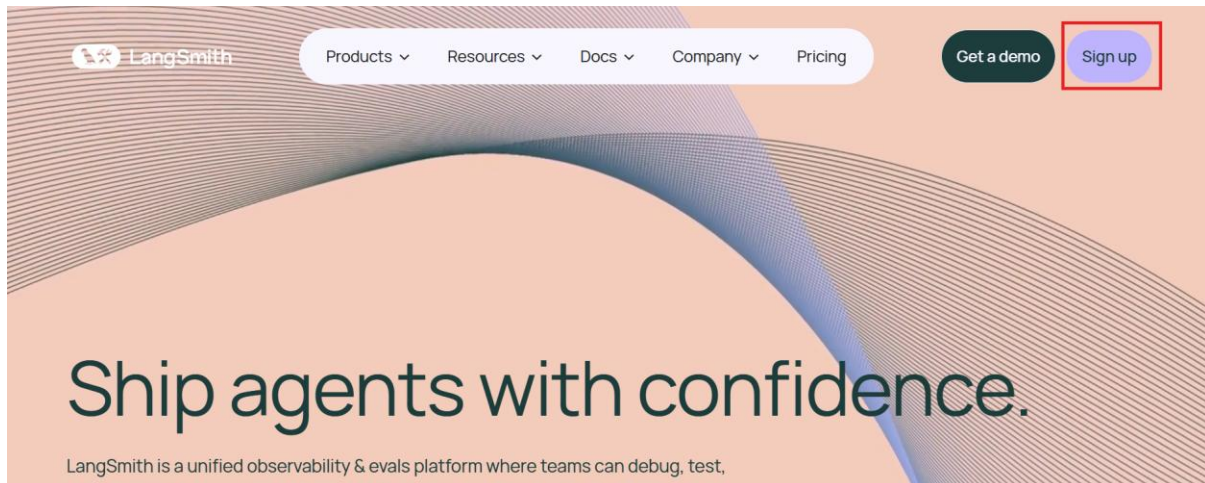
4. Run the test script:
5. `python test_api.py`

Runs comprehensive tests against your deployed application.

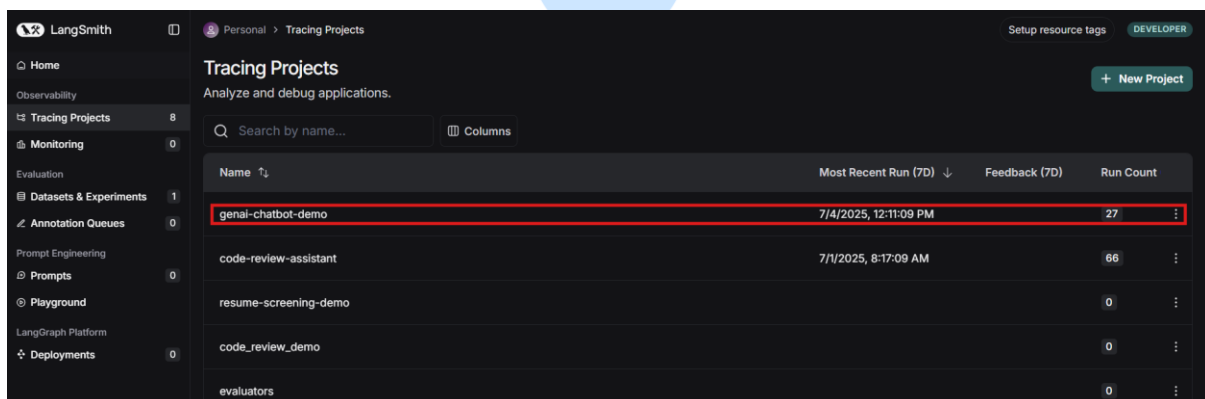
Monitoring and Logging

Step 29: Access LangSmith Dashboard

1. **Open LangSmith:** Go to <https://smith.langchain.com/>



2. **Login** with your account
3. **Navigate to your project:** Look for "genai-chatbot-demo"



4. **View traces:** You should see traces from your API calls

The screenshot shows the 'genai-chatbot-demo' dashboard. At the top, there are tabs for 'Runs', 'Threads', 'Alerts', and 'Setup'. Below the tabs, there's a filter section with '1 filter' and 'Last 7 days'. The main table lists individual AI runs with columns for Name, Input, Output, Error, Start Time, and Latency. The right sidebar contains 'Stats' (Run Count: 25, Total Tokens: 18,409, Median Tokens: 696, Error Rate: 0%, % Streaming: 0%, Latency: P50: 3.51s, P99: 5.15s) and 'Filter Shortcuts'.

Name	Input	Output	Error	Start Time	Latency
ChatGoogleGenerativ...	human: What's the dif...	ai: Think of it like ...		7/4/2025, 12:11:09 ...	3
ChatGoogleGenerativ...	human: Can you expl...	ai: Okay, imagine ...		7/4/2025, 12:11:04 ...	3
ChatGoogleGenerativ...	human: What is artific...	ai: Artificial intelli...		7/4/2025, 12:10:58 ...	4
ChatGoogleGenerativ...	human: Hello! How ar...	ai: I am doing wel...		7/4/2025, 12:10:56 ...	0
ChatGoogleGenerativ...	human: Hello, how ar...	ai: I am doing wel...		7/4/2025, 11:39:42 ...	0
ChatGoogleGenerativ...	human: Hello, how ar...	ai: I am doing wel...		7/4/2025, 11:39:17 ...	0
ChatGoogleGenerativ...	human: What's the dif...	ai: Think of it like ...		7/4/2025, 11:32:51 ...	3
ChatGoogleGenerativ...	human: Can you expl...	ai: Imagine you're...		7/4/2025, 11:32:45 ...	2
ChatGoogleGenerativ...	human: What is artific...	ai: Artificial intelli...		7/4/2025, 11:32:38 ...	4

The screenshot shows the 'genai-chatbot-demo' dashboard with a detailed view of a specific AI run. The left sidebar shows the 'Runs' tab. The main area displays the 'TRACE' for a 'gemini-2.0-flas...' model, which took 3.50s. The right sidebar shows the 'ChatGoogleGenerativeAI' model details, including the 'Run', 'Feedback', and 'Metadata' tabs. The 'Input' section shows a conversation between a HUMAN and an AI.

Name	Input
ChatGoogleGenerativ...	human: What's the dif...
ChatGoogleGenerativ...	human: Can you expla...
ChatGoogleGenerativ...	human: What is artifici...
ChatGoogleGenerativ...	human: Hello! How ar...
ChatGoogleGenerativ...	human: Hello, how are...
ChatGoogleGenerativ...	human: Hello, how are...
ChatGoogleGenerativ...	human: What's the dif...
ChatGoogleGenerativ...	human: Can you expla...
ChatGoogleGenerativ...	human: What is artifici...

Shows you detailed information about each AI interaction, including:

- Response times
- Token usage
- Error rates
- Conversation flows

Step 30: Check Cloud Run Logs

Check the logs to see what went wrong

```
gcloud logging read "resource.type=cloud_run_revision AND  
resource.labels.service_name=genai-chatbot" --limit 50
```

Shows you application logs from Cloud Run, helpful for debugging and monitoring.

Step 41_init_: Monitor Cloud Run Metrics

1. **Open Google Cloud Console:** Go to <https://console.cloud.google.com/>
2. **Navigate to Activities**
3. **View metrics:** Check requests, latency, and errors

Provides insights into your application's performance and usage patterns.

